PyAutoRoute Tutorial

An introduction to PyAutoRoute

D. Q. McDonald

Note: This tutorial is a work in progress. Some sections may be incomplete or subject to change.

Contents

1. Quick Start

From a finished schematic to a placed and routed board in five steps:

- **Draw your schematic in KiCad** and run the Electrical Rules Checker (*Inspect* → *Electrical Rules Checker*). Fix any errors before continuing.
- **Assign footprints.** Use KiCad's footprint assignment tool, or let `pyautoroute-assign` fill them in automatically from a personal preference database:

  ```
  pyautoroute-assign MyBoard.kicad_sch
  ```

- **Push the schematic to a PCB.** In KiCad open the PCB editor and choose *Tools* → *Update PCB from Schematic* (`F8`). Add an `Edge.Cuts` board outline that generously encloses all footprints (at least 2 mm of margin), then save.
- **Place and route with PyAutoRoute.** The single command below places the footprints, then routes all nets:

  ```
  pyautoroute MyBoard.kicad_pcb --place --place-time 60 --routing-
      time 60
  ```

  A graphical interface is also available (`pyautoroute-gui`) if you prefer to watch the optimiser converge in real time or set per-footprint constraints (lock a connector, pull a LED to the edge, etc.).
- **Review the result in KiCad.** Open `MyBoard_placed_routed.kicad_pcb`, run *Inspect* → *Design Rules Checker*, and expect zero clearance violations. PyAutoRoute also prints a self-check summary:

  ```
  connections:  42/42 routed (100%)
  self-check:   clean (0 clearance violations)
  ```

The rest of this tutorial walks through each step in detail using a worked example — a selectable-clock board for a Z80 retrocomputer.

2. Introduction

PyAutoRoute is a Python tool that automatically places and routes two-layer KiCad PCBs without requiring the `pcbnew` Python bindings — it reads and writes the `.kicad_pcb` s-expression format directly, so it runs in any normal Python environment.

Given a board with footprints placed and nets assigned, PyAutoRoute produces a routed copy that is DRC-clean by construction: clearance to other tracks, pads, vias, and the board edge is enforced on an inflated routing grid, so the router can never produce a short or a clearance violation.

It can also *place* the footprints automatically before routing, using a simulated-annealing pass that minimises rats-nest length while keeping component bodies apart.

This tutorial walks through routing a small but realistic board: a selectable-clock circuit for a Z80 retrocomputer, offering three clock sources controlled by a jumper.

3. The Tutorial Circuit

The first stage is to create a schematic. For this tutorial we'll be using a circuit (Figure 1) that implements three clock sources for a Z80-based computer:

1. **555 Timer clock** — a TLC555xP astable oscillator whose frequency is set by a potentiometer (`RV1`) and timing capacitors.
2. **4 MHz crystal oscillator** — a Pierce oscillator built from two 74HC14 Schmitt-trigger inverters and a 4 MHz crystal (`Y1`).
3. **Manual (single-step) clock** — a debounced pushbutton (`SW1`) for stepping through instructions one cycle at a time.

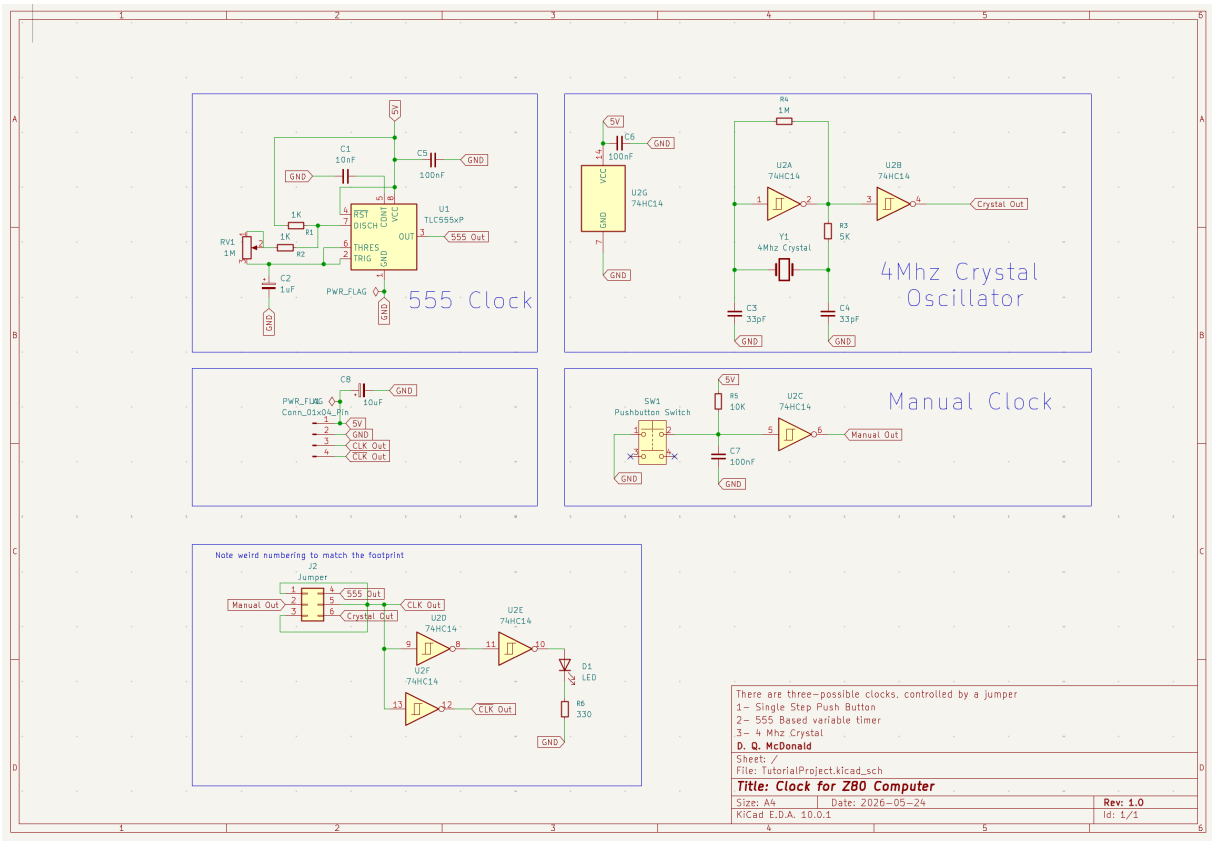

Figure 1: KiCad schematic — Clock for Z80 Computer.

Once your schematic is complete you should use the Electrical Rules Check to ensure that it is complete. Then you can assign footprints using the built-in tool in KiCad. This can also be a tiresome task and the next section describes a tool that provides some help with automating that.

Assigning footprints with `pyautoroute-assign`

Before a schematic can be turned into a PCB layout, every symbol needs a *footprint* — the physical land pattern that will be manufactured on the board. PyAutoRoute includes a companion tool, `pyautoroute-assign`, that fills in empty `Footprint` fields in a `.kicad_sch` file automatically, using a personal preference database you configure once.

Footprints are assigned by component prefix (`R`, `C`, `CP`, `U`, ...) and, for ICs, by the symbol's

`Value` field. A quick preview before any file is written is available with `--dry-run`:

```
# Preview what would be assigned
pyautoroute-assign TutorialProject.kicad_sch --dry-run

# Assign footprints using your preferences
pyautoroute-assign TutorialProject.kicad_sch
```

On first use, run `--init-prefs` to create the preference file and `--rebuild-index` to scan your KiCad library. See the *Assigning footprints* section of the `README` for the full option reference.

4. Prerequisites

- **KiCad 6 or later** — to view and edit the schematic and PCB files.
- **Python 3.10+** with `numpy`, `scipy`, and `shapely` $\geq$ `2.0`.
- **PyAutoRoute** installed:

  ```
  pip install -e /path/to/PyAutoRoute
  ```

- The tutorial project files in the `TutorialProject/` subdirectory alongside this document.

Verify the installation:

```
pyautoroute --version
```

5. Starting Point: the Unrouted Board

Open `TutorialProject/TutorialProject.kicad_pcb` in KiCad. You will see the footprints arranged roughly in functional groups (Figure 2), with thin *ratsnest* lines indicating the connections that need to be routed.

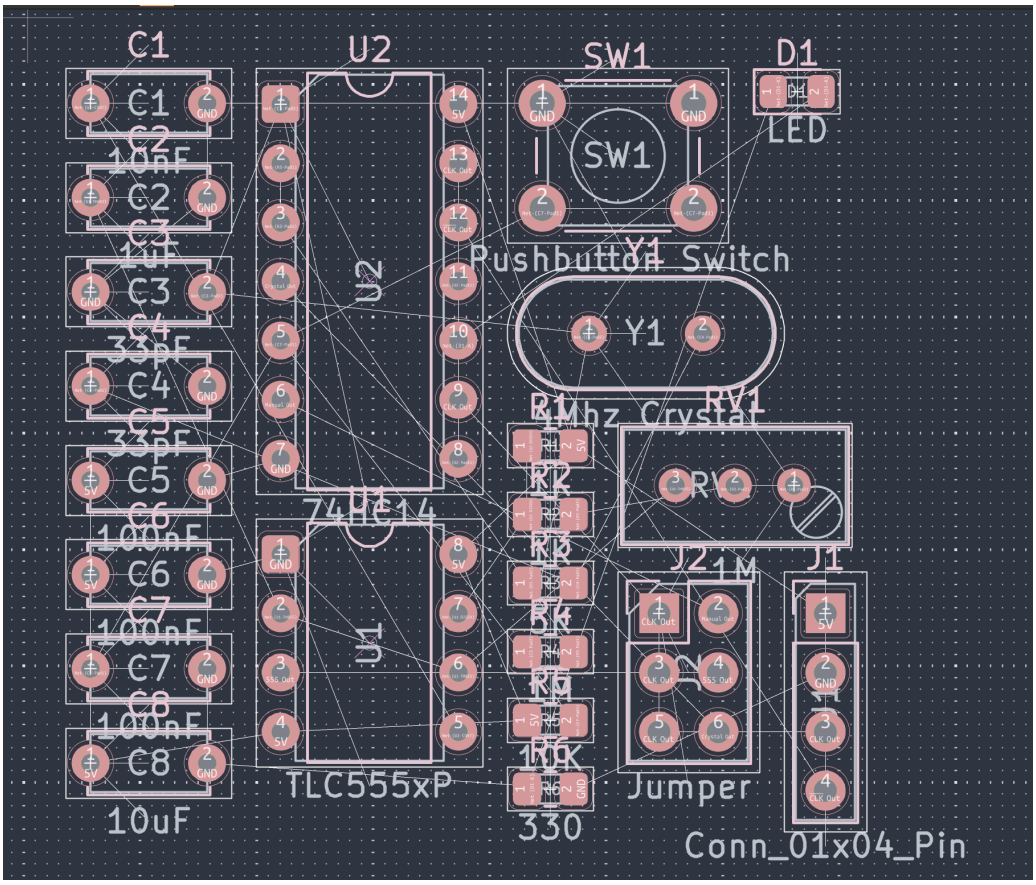

Figure 2: Initial PCB layout — footprints placed, no tracks yet. The pink lines are the ratsnest (unrouted connections).

At this stage:

- All footprints have been placed manually in approximate functional groups.
- Nets are assigned (exported from the schematic via *File* → *Export* → *Netlist*).
- No tracks exist yet — the board is purely a placed netlist.

6. Adding a Board Outline

Before routing, KiCad needs an `Edge.Cuts` outline so the router knows the board boundary. Figure 3 shows the board after a rectangular outline has been drawn around the components. If you are intending to use PyAutoRoute to place components, draw a board outline that is generously sized around the components to give them room to move. You are free to move any components you want into a specific place and lock them — PyAutoRoute will honour that. The placer may not use all the space, but you can easily resize the board once routing and placement are done if needed. Of course, if you have a specific board size or shape requirement, draw that outline now and pass `-keep-outline` (or enable *Keep board outline* in the GUI Advanced Settings, Section 8.3) to constrain placement inside it.

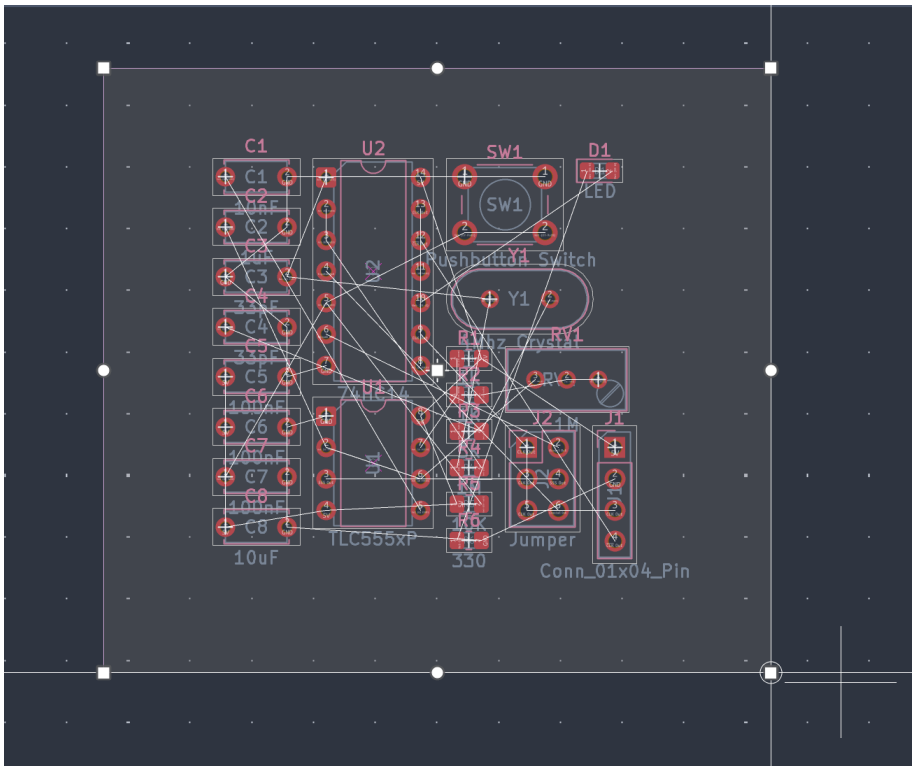

Figure 3: Board with `Edge.Cuts` outline added. PyAutoRoute keeps all tracks inside this boundary.

To draw the outline in KiCad:

1. Switch to the **Edge.Cuts** layer.
2. Use *Place* → *Draw Rectangle* to draw a rectangle that comfortably encloses all footprints (leave at least 2 mm of margin).
3. Save the board (`Ctrl+S`).

Alternatively, PyAutoRoute can generate the outline automatically during a `-place` run; the manual step is only needed when routing a pre-placed board.

7. Routing with PyAutoRoute

7.1 Quick greedy route

The simplest invocation routes the board in a single greedy pass — no annealing, just the fastest possible complete route:

```
pyautoroute TutorialProject/TutorialProject.kicad_pcb
```

This writes `TutorialProject_routed.kicad_pcb` alongside the input. Open it in KiCad to inspect the result.

7.2 Optimised route

For a better result, add a time budget for simulated-annealing optimisation. The annealer rips up and reroutes connections to reduce total track length and via count:

```
pyautoroute TutorialProject/TutorialProject.kicad_pcb \
    --routing-time 60
```

A 60-second budget is usually enough for a board of this size. Watch the live progress output:

```
[  0.1s] greedy route: 42/42 connections (100%)
[  0.2s] annealing ...
[  0.2s]  1000 iters  59s rem  E= 312.4  best= 308.1  acc= 34%
[  0.5s]  2000 iters  58s rem  E= 289.7  best= 285.3  acc= 22%
...
[ 60.1s] writing routed board ...
  connections:  42/42 routed (100%)
  wirelength:   274.3 mm
  vias:         8
  self-check:   clean (0 clearance violations)
```

7.3 Place and route

If you want PyAutoRoute to also arrange the footprints before routing, use `-place`:

```
pyautoroute TutorialProject/TutorialProject.kicad_pcb \
    --place --place-time 30 --routing-time 60
```

The placer uses simulated annealing to minimise rats-nest length while keeping component bodies at least `-place-buffer` apart (default: derived from the design-rule clearance). It also respects locked footprints (fixed obstacles) and footprints marked `Autoroute-edge` (pulled to the board boundary — useful for connectors).

7.4 Common options

Option	Effect
`-grid 0.2`	Finer routing grid (mm); better coverage, slower
`-routing-time 120`	2-minute annealing budget
`-runs 4`	Best-of-4 independent routing runs
`-via-weight 5`	Penalise vias more heavily (fewer layer changes)
`-ground-plane`	Add a GND copper pour on B.Cu after routing
`-place-only`	Place footprints without routing

8. Using the GUI

PyAutoRoute includes a graphical interface that exposes the same functionality as the command line in a more interactive form — useful for exploring settings, setting per-footprint constraints, and watching the placement and routing optimisers converge in real time.

Launch it by opening a `.kicad_pcb` file from the File menu, or from the terminal:

```
pyautoroute-gui
```

8.1 The main window

Figure 4 shows the GUI after opening the tutorial board. The window is divided into three areas:

- **Left panel** — mode selector (Route only / Place + Route / Place only), routing and placement parameters (grid, time budget, runs, existing-route handling), and action buttons (*Run*, *Apply to Project*, *Save As…*, *Advanced…*).
- **Centre canvas** — an interactive view of the board. Click any footprint to open the constraint context menu. The view selector at the bottom switches between Initial, Current, Best, Overall Best, Rats-nest, and Energy heat-map views.
- **Right panel** — live run statistics (phase, elapsed time, iteration count, energy, acceptance rate, temperature, routed/unrouted counts, self-check result) and an energy convergence graph that updates as the annealer runs.

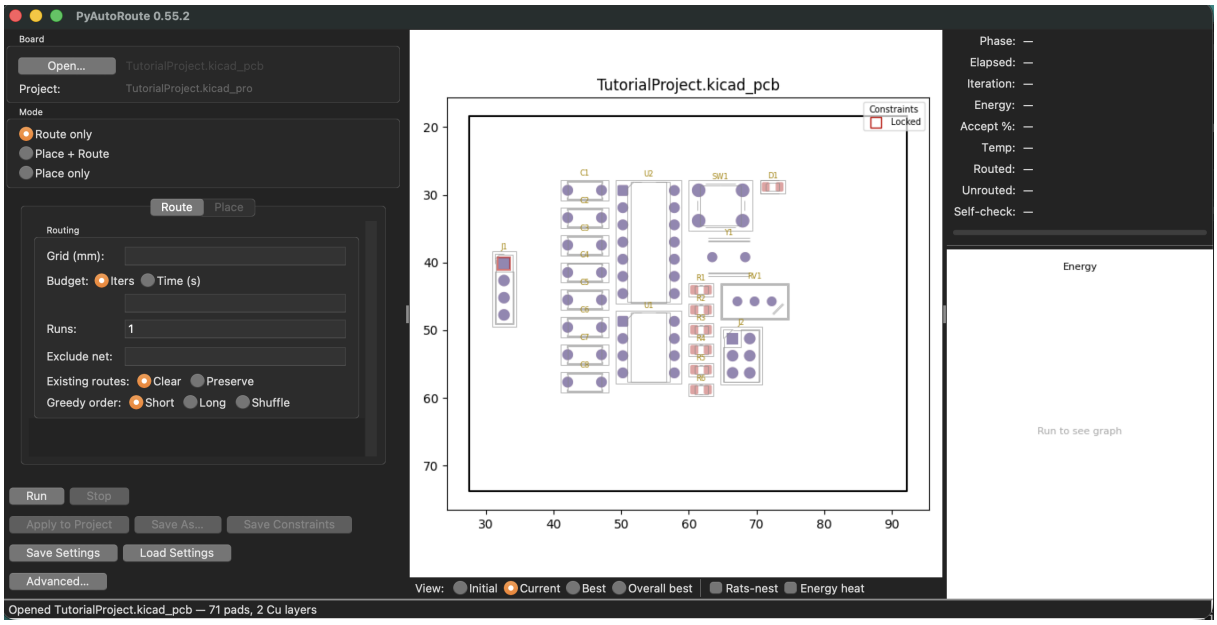

Figure 4: The PyAutoRoute GUI after opening the tutorial board. Controls are on the left, the board canvas in the centre, and live run statistics on the right.

8.2 Setting footprint constraints

Right-clicking any footprint on the canvas opens the constraint menu (Figure 5), with four options. Note that the right-click menu is only available when the canvas is in **Initial View** mode — select it from the view selector at the bottom of the canvas before right-clicking.

- **Edge** — attach the footprint to a board edge (*Left*, *Right*, *Top*, *Bottom*, or *None*). The part is then pulled to that boundary during placement and oriented flat against it.
- **Lock** — freeze the footprint's position so the placer treats it as a fixed obstacle.
- **Overlap OK** — allow this footprint's body to overlap others (e.g. a shield sitting over the board).
- **Decoupling cap** — pull a capacitor toward its associated IC during placement.

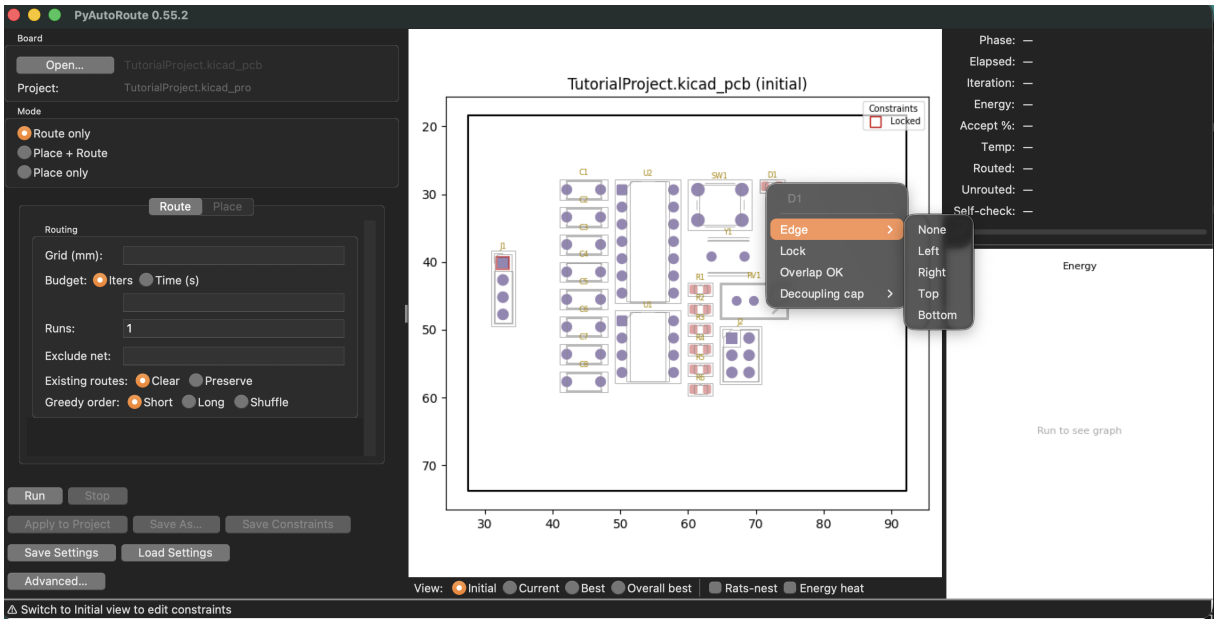

Figure 5: Right-clicking `D1` (the LED) opens the constraint menu. Here the *Edge* submenu is open, ready to attach the LED to a specific board boundary.

For this tutorial, set `D1` to prefer the right edge (*Edge* → *Right*). The canvas immediately shows an orange arrow on `D1` pointing toward the right boundary (Figure 6), confirming the constraint is active.

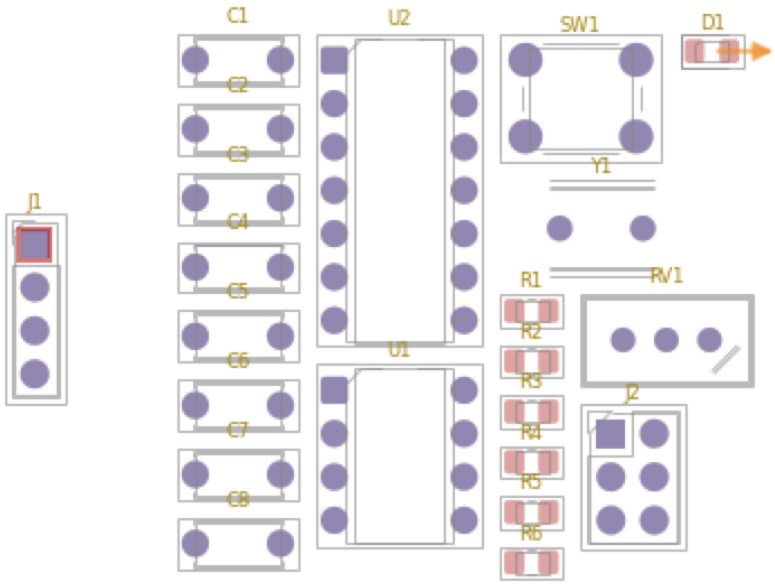

Figure 6: `D1` with an *Edge* = *Right* constraint applied. The orange arrow indicates the target edge; the placer will pull the LED to the right boundary and orient it flat against it.

To lock the output connector `J1` in its current position, right-click it and choose *Lock*. Locked footprints are shown with a distinctive highlight in the canvas (Figure 7) and listed in the

constraints legend (top-right of the canvas).

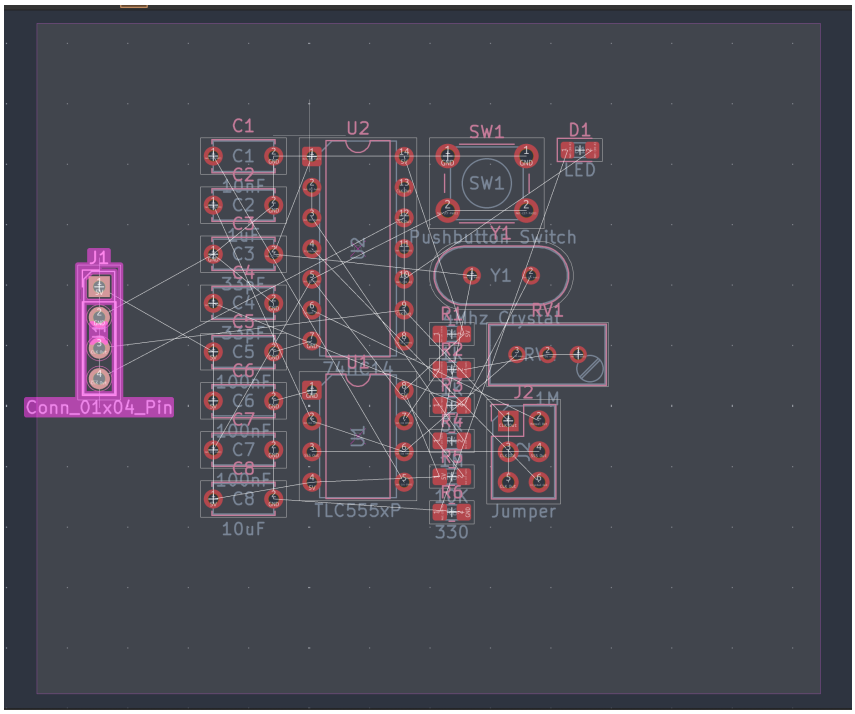

Figure 7: `J1` locked in place. The placer will treat it as a fixed obstacle; all other footprints are arranged around it.

8.3 Advanced settings

Click *Advanced…* to open the full settings dialog (Figure 8). This exposes the placement and routing energy weights, annealing schedule temperatures, post-processing options (silk labels, ground plane, mounting holes), and the **Keep board outline** toggle.

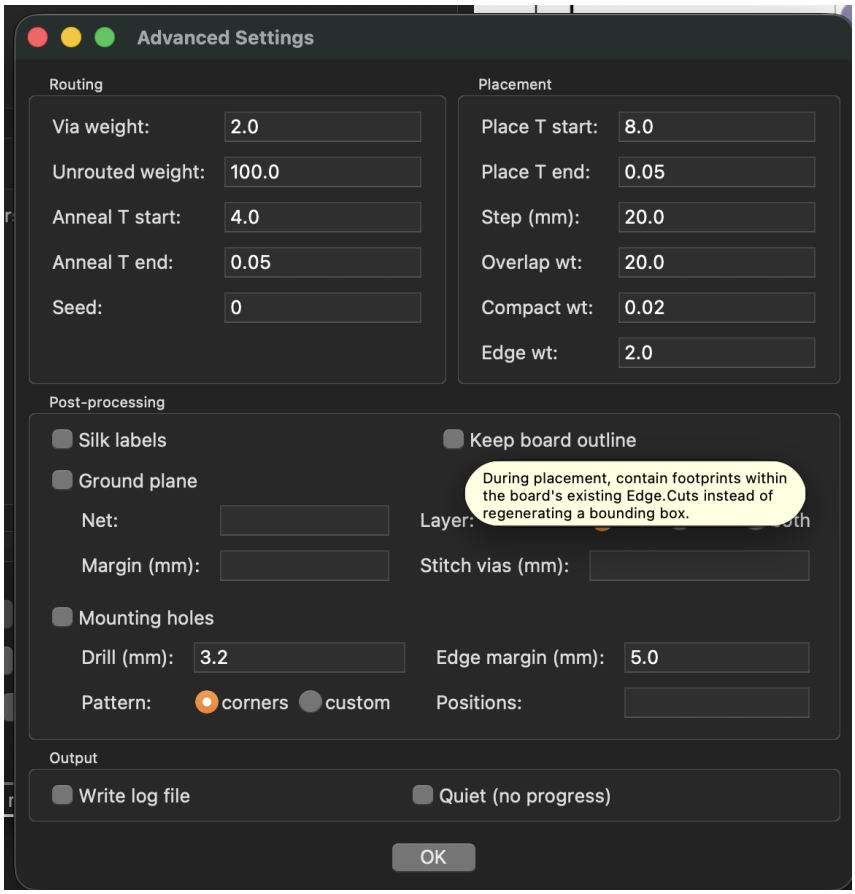

Figure 8: The Advanced Settings dialog. The *Keep board outline* option (highlighted) constrains placement to the existing `Edge.Cuts` shape instead of regenerating a bounding rectangle — useful when the board has a fixed mechanical outline.

8.4 Running placement

Select **Place only** mode, set a time budget (60 s is a good starting point) and a run count of 3, then click *Run*. The annealer starts immediately; the energy graph on the right updates in real time as the temperature cools and the placement converges.

Figure 9 shows the result after three 60 s runs. The footprints are compactly arranged, `D1` has been pulled to the right boundary as requested, `J1` is in its locked position, and the self-check reads **PASS**.

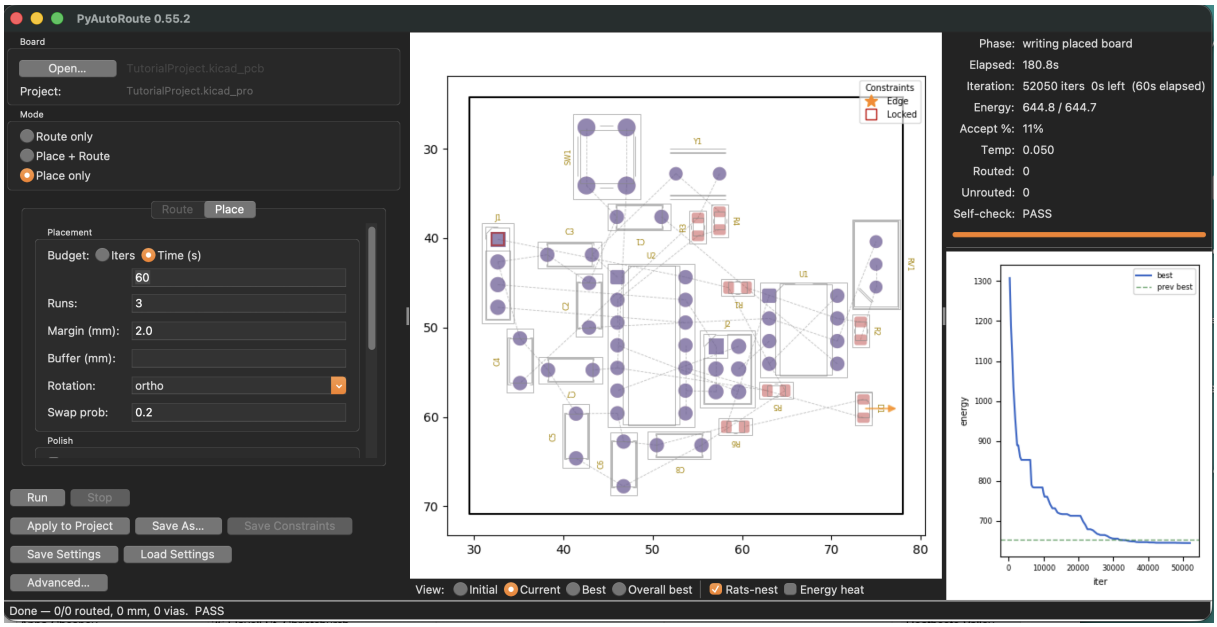

Figure 9: After three placement runs of 60 s each (180 s total). The energy graph shows the annealer converging; the status panel confirms 0 unrouted connections and a clean self-check.

8.5 Running the router

Switch to **Route only** mode (the placed board is already in memory), set a 30 s time budget, and click *Run* again. The router completes the greedy pass and then anneals to reduce track length and via count.

Figure 10 shows the finished board: all 53 connections routed, 13 vias, 560 mm total track length, and Self-check: **PASS**. Click *Save As...* to write the result to a `.kicad_pcb` file, or *Apply to Project* to overwrite the project board in place.

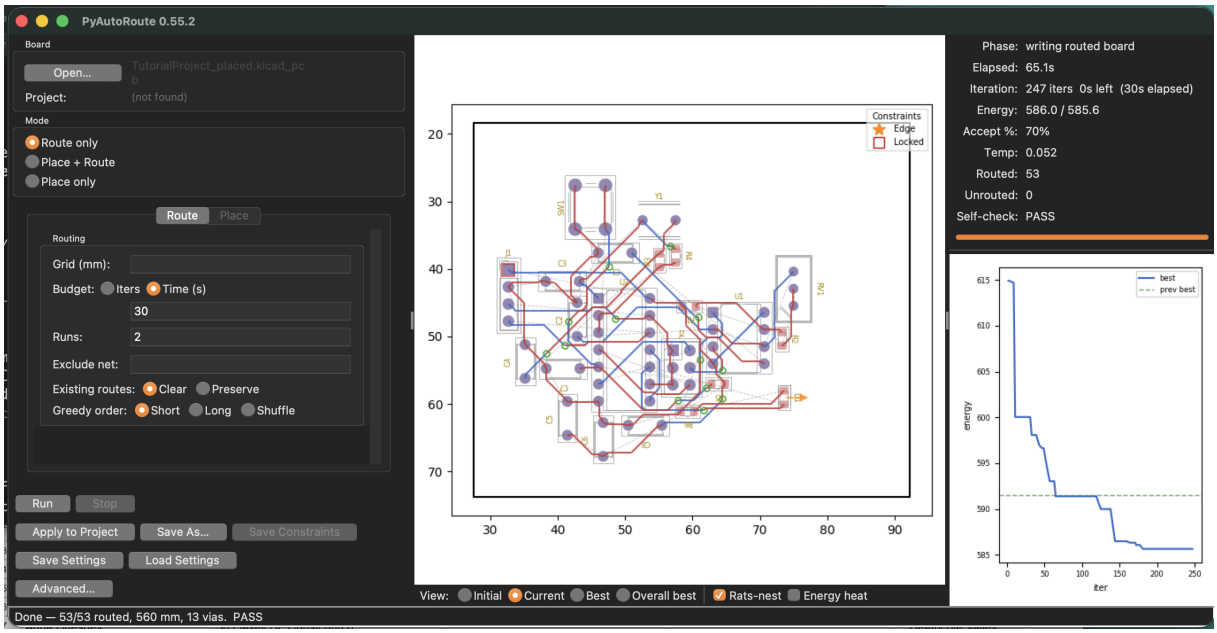

Figure 10: The completed routed board. All 53 connections are routed (0 unrouted), with 13 vias and a clean self-check. The energy graph shows the annealing optimisation converging over the 30 s budget.

9. Reviewing the Result

Open the routed board in KiCad (*File* → *Open*). Check:

1. **DRC** — run *Inspect* → *Design Rules Checker*. Expect zero clearance violations; any "unconnected" items match the connections PyAutoRoute reported as unrouted.
2. **Track inspection** — use the 3D viewer (*View* → *3D Viewer*) to visually confirm track routing and layer usage.
3. **Unrouted connections** — if any remain, try a finer `-grid`, a longer `-routing-time`, or more `-runs`.

PyAutoRoute also runs its own internal geometry self-check and reports the result on the command line:

```
self-check:   clean (0 clearance violations)
drill-check:  clean (0 hole-to-hole violations)
```

A non-zero exit code means a violation was found — this should not happen on a correctly set-up board, but the check provides an extra safety net before you open KiCad.

10. Next Steps

- **Settings file** — save your preferred options so you don't have to retype them:

```
pyautoroute TutorialProject.kicad_pcb \
    --grid 0.2 --routing-time 60 --write-config
# creates TutorialProject.ini; next time just run:
pyautoroute TutorialProject.kicad_pcb
```

- **Ground plane** — add `-ground-plane` to flood B.Cu with GND copper after routing. PyAutoRoute will call `kicad-cli` to fill the zone if it is installed.
- **KiCad plugin** — install the plugin (`pyautoroute-install-plugin`) to run PyAutoRoute directly from the KiCad toolbar without leaving the editor.
- **Full documentation** — see `README.md` and `docs/architecture.md` in the PyAutoRoute repository for the complete option reference and algorithm details.

11. Frequently Asked Questions

Some connections are still unrouted after routing. What should I try?

Unrouted connections mean the A* search could not find a path — usually because the routing grid is too coarse to thread between closely-spaced pads, or the annealing budget ran out before a global solution emerged. Try these in order:

1. **Finer grid** — halve the grid pitch:

```
pyautoroute board.kicad_pcb --grid 0.1
```

   A finer grid gives the router more room to manoeuvre around dense pad clusters, at the cost of a slower run.

2. **Longer annealing budget** — the rip-up/reroute loop needs time to untangle conflicting connections:

```
pyautoroute board.kicad_pcb --routing-time 300
```

3. **Multiple runs** — annealing is stochastic; a different seed often finds a route the first seed missed:

```
pyautoroute board.kicad_pcb --routing-time 60 --runs 5
```

4. **Check pad clearances** — if two pads sit closer together than the design-rule clearance, no router can pass copper between them. Open the board in KiCad and run DRC on the *unrouted* board to catch physical conflicts before routing.

The routed board has too many vias. How do I reduce them?

Each via costs `-via-weight` mm of equivalent track length in the annealing energy (default 2.0). Raising this value makes the optimiser treat vias as more expensive, so it works harder to stay on one layer:

```
pyautoroute board.kicad_pcb --routing-time 60 --via-weight 8
```

Very high values (above 10) can make some nets unroutable if the only path requires a layer change, so increase gradually. A complementary approach is to prefer `F.Cu` with the default layer bias (already built in) and use a finer grid so more single-layer paths become available.

Routing is very slow on my board. How can I speed it up?

Two knobs help:

- **`-search-margin MM`** — bounds each connection's A* search to a box around its endpoints, grown by `MM` on every side. The router widens the box and retries if it cannot find a route, ultimately falling back to the whole grid. This dramatically speeds up the rip-up/reroute loop on large or sparse boards:

```
pyautoroute board.kicad_pcb --routing-time 120 --search-margin 10
```

- **Coarser grid** — a 0.2 mm or 0.25 mm grid is significantly faster than 0.1 mm and is sufficient for most hobby boards with 0.2 mm clearance rules. Use `pyautoroute-tune` to find the best grid for your board automatically.
- **Parallel runs** — combine `-runs N` with `-jobs 0` to use all CPU cores, keeping total wall-clock time the same while exploring more seeds:

```
pyautoroute board.kicad_pcb --routing-time 60 --runs 8 --jobs 0
```

Components are overlapping after `--place`. What is wrong?

The placement annealer uses the **courtyard layer** (`F.CrtYd` / `F.Courtyard`) to determine each footprint's physical body extent. If a footprint in your KiCad library is missing a courtyard, the placer falls back to the pad bounding box, which can be much smaller than the actual body — a common situation with round electrolytic capacitors, whose two pads span only the lead pitch while the body extends well beyond them.

To fix this:

- Open the footprint editor in KiCad and add a `F.CrtYd` rectangle or circle that tightly encloses the physical body.
- Alternatively, increase `-place-buffer` to add extra clearance around all footprints, which compensates for a missing or undersized courtyard at the cost of a larger layout:

```
pyautoroute board.kicad_pcb --place --place-buffer 2.0
```

How do I keep a connector at the board edge?

Add a footprint property named `Autoroute-edge` with value `any` (nearest edge), or `left` / `right` / `top` / `bottom` to pin a specific side. In KiCad: *Footprint Properties* → *Fields* → +, name `Autoroute-edge`, value `right`.

The placer then pulls the footprint to that edge *and* orients it flat against the boundary, so all its pins lie near the board perimeter. The pull strength is controlled by `-place-edge-weight` (default 2.0; higher pulls harder). Unlike locking, the footprint is still free to slide along the edge while the rest of the layout optimises around it.

How do I stop a specific component from moving during placement?

Lock the footprint in KiCad (*right-click* → *Lock*, or press `L`). KiCad stores this as a `(locked yes)` attribute in the `.kicad_pcb` file, which PyAutoRoute reads and treats as a fixed obstacle. Locked footprints are listed on startup so you can confirm what is pinned before a run.

This is useful for mounting holes, edge-mount connectors, or any part that must occupy an exact mechanical position.

Where does PyAutoRoute write its output, and what are the file names?

PyAutoRoute never modifies the input file (unless you pass `-in-place`). Output naming follows the mode:

Mode	Output filename
Route only (default)	`INPUT_routed.kicad_pcb`
Place then route (`-place`)	`INPUT_placed_routed.kicad_pcb`
Place only (`-place-only`)	`INPUT_placed.kicad_pcb`

Use `-o PATH` to override the output path entirely.

How do I re-run with the same settings without retyping them?

Write the current settings to an INI file with `-write-config`:

```
pyautoroute board.kicad_pcb --grid 0.15 --routing-time 120 \
    --via-weight 5 --write-config
```

This creates `board.ini` alongside the board. On every subsequent run, PyAutoRoute auto-loads that file — so you can just type:

```
pyautoroute board.kicad_pcb
```

Command-line options always override the file, so you can still do one-off experiments without editing it.

Can I route a board that I have already partially routed by hand?

Yes. Pass `-existing-routes preserve`:

```
pyautoroute board.kicad_pcb --existing-routes preserve
```

PyAutoRoute keeps all existing copper, works out which connections it already satisfies, and routes only the remainder — treating the existing tracks as fixed obstacles. This lets you hand-route critical signals first and delegate the rest to the autorouter.

The default is `-existing-routes clear`, which strips all tracks before routing so re-running never doubles copper.

Does PyAutoRoute support more than two copper layers?

No. The current router uses exactly two layers: `F.Cu` (front) and `B.Cu` (back). Four-layer or six-layer boards are not supported. If your design requires inner layers for power planes or high-density routing, you will need a different tool for those nets; PyAutoRoute can still handle the signal routing on the two outer layers if the inner-layer copper is treated as a fixed obstacle.

Can it handle copper pours and ground planes?

PyAutoRoute does not *route* into copper pours, but it can **add a GND pour** after routing with `-ground-plane`:

```
pyautoroute board.kicad_pcb --ground-plane \
    --ground-plane-layer B.Cu --stitch-vias
```

This emits a zone boundary following the board outline on the chosen layer. If `kicad-cli` is installed, PyAutoRoute fills the zone automatically; otherwise open the board in KiCad and run *Edit* → *Fill All Zones* (`B`).

Existing filled copper zones in the input board are detected and excluded from routing — their net is not routed and the filled polygon is not treated as an obstacle (KiCad fills them after the fact).

Will PyAutoRoute handle impedance-controlled or high-speed signals correctly?

Not automatically. PyAutoRoute minimises total track length and via count but has no knowledge of:

- **Controlled impedance** — track width for a target impedance on a given stack-up must be set manually (use a fixed net-class width in KiCad; PyAutoRoute respects net-class rules).
- **Matched-length pairs** — the router does not length-tune single-ended signals. Differential pairs (`-diff-pairs`) are routed coupled with controlled gap, which helps impedance, but skew trimming is not performed.
- **Crosstalk, return-path continuity, EMC** — none of these are modelled.

For low-frequency hobby boards (audio, microcontrollers, retrocomputer logic) these limitations rarely matter. For RF, high-speed digital (DDR, USB 3, PCIe), or safety-critical designs, use PyAutoRoute as a starting point and review the critical nets manually before fabrication.